

Token Usage in the Processminer Skill System

June 12, 2026

CONTENTS

Token Usage in the Processminer Skill System	
A Deep-Dive Analysis and Optimization Plan	
1. Executive Summary	
2. What the Measured Data Says	
3. The Karpathy LLM-Wiki Pattern as a Token Strategy	
Red lines no optimization may cross	
4. Lever 1 — Skill Design: Stop Hoarding the Document in the Transcript	
5. Lever 2 — Tool-Result Diet in the MCP Server	
6. Lever 3 — Static Prompt Diet (Paid by Every Session, Every Call)	
7. Lever 4 — The Gemini Track Needs Its Own Structural Pass	
8. Lever 5 — Fix the Measurement First	
9. What This Plan Deliberately Does Not Do	
10. Priority Order and Expected Impact	

Token Usage in the Processminer Skill System

A Deep-Dive Analysis and Optimization Plan

Date: 2026-06-12 · **Scope:** Both session backends (Claude CLI / MCP and Gemini), the 28 skills, the shared CORE system prompt, and the schema/process-JSON context model ·

Constraint: Every recommendation adheres to the Karpathy LLM-Wiki pattern and the schema / process-JSON principles.

1. Executive Summary

The dominant token cost in Processminer is not what the model writes — it is what gets **re-played**. The runtime store records, for a single dogfood test run, **24.2 million cache-read tokens against just 3,084 fresh input tokens**: the model re-reads an enormous, ever-growing context on every internal model call. Output tokens are outnumbered by context replay roughly 100 : 1.

The cost shape is:

(static prompt + hoarded element bodies + accumulated tool results) × number of model calls per session

Three levers dominate, and all three are not merely compatible with the Karpathy LLM-Wiki pattern — they are *returns to it*:

1. **Skill design** — `foundational-run` (and, in milder form, `document-ingest` and the specialists' orientation phases) loads every element body up front and holds it in the transcript for the whole run. This is both the largest measured cost (7.7 M cache-read tokens in one run) and a violation of the pattern: it promotes the transcript to a shadow source of truth.
2. **Tool-result diet** — the MCP server echoes full elements back on every write, returns full provenance maps on collection reads, and pretty-prints every result (+40–60 %). The stored JSON keeps full fidelity; the view handed to the model does not need it.
3. **Gemini track structure** — the entire 80 KB schema (~20,000 tokens) plus ~26 KB of tool declarations are re-sent on **every** `generateContent` call, and the per-turn document map sits inside the system instruction where it defeats prefix caching.

A fourth, prerequisite finding: **the recorded per-skill dollar costs are inflated roughly 30×** by an accounting bug (the CLI's *cumulative* `total_cost_usd` is summed per turn). The token counts are trustworthy; the dollar figures are not. Fix the measurement first so every other improvement is verifiable.

Estimated impact of the full plan on a typical long skill run: **60–80 % reduction in replayed context**, with the foundational-run redesign alone accounting for the majority.

2. What the Measured Data Says

Recorded in the runtime store (`data/runtime/dogfood-test-run-2.json`, Claude track, `claude-sonnet-4-6`):

Skill	Output tokens	Cache-read tokens	Cache-creation tokens
foundational-run	59,767	7,700,601	204,917
free-chat	60,993	6,617,919	437,257
document-ingest	42,282	6,074,192	125,488
council-review	9,230	1,031,056	15,670
dtp-summary	3,228	989,399	27,587
run-lint	11,127	882,745	38,827
dtp-compare	4,476	928,751	6,153

Reading the table: cache-read tokens are the whole session context replayed on every model call inside every tool round-trip. `foundational-run` ran ~23 minutes interactively; at ~150 K tokens of context per call, ~50 calls explain the 7.7 M replay. The levers are therefore: *shrink what enters the context* and *shorten how long it is carried*.

The accounting bug: the recorded cost for foundational-run is \$138.37, but its token counts at Sonnet 4.6 rates compute to roughly \$4–5. The Claude CLI's `total_cost_usd` on the `result` event is **cumulative per session**; `src/app/api/session/route.ts` records it every turn and sums, inflating quadratically. (Flagged as a separate fix task.)

3. The Karpathy LLM-Wiki Pattern as a Token Strategy

The pattern is not just an audit/trust architecture — it is itself a token strategy. The wiki JSON is the persistent memory, the context window is a disposable workbench, and the schema is the contract between them. Every major waste point found is a place where the implementation strays from the pattern; every fix is a return to it.

LLM-Wiki principle	Token discipline it implies
The wiki JSON is the <i>only</i> durable memory	The transcript must never hoard wiki content “for later” — re-read on demand
Context is a derived view of the wiki	Abridged views (document map, schema slice, id + title lists) are computed on read, shown to the model, and discarded — never persisted
Runtime state lives above the wiki (R9)	Session/cursor state in the runtime store makes transcripts disposable — sessions can be cycled freely
Provenance is the trust foundation	Full fidelity lives in the stored JSON only; ephemeral views may omit evidence, the document never does
The schema is the single contract	Schema views are sliced from <code>schema/process-schema.json</code> at injection time, never forked into prose

RED LINES NO OPTIMIZATION MAY CROSS

1. Never trim, summarize, or restructure provenance/evidence **in the stored JSON** — only ephemeral views get abridged.
2. Never persist a compacted or abridged representation anywhere (no `document-map.json` sidecar, no cached schema slice on disk). Derived views must be obviously derived and re-generated on every read.
3. Never let a skill instruct the model to treat transcript content as current truth.
4. Session-cycling state goes in the runtime store only — no “resume context” blobs anywhere near `wiki/`.
5. All writes stay behind the schema-validated typed writers. Nothing in this plan changes the mutation path.

4. Lever 1 — Skill Design: Stop Hoarding the Document in the Transcript

The finding. `foundational-run/SKILL.md` Step 1 instructs: take the snapshot, then `expand-Element` **every current-state element body** and *“keep those bodies as your working copy for the whole run — do not re-`expandElement` an unchanged element later.”* For a 150 KB process this parks ~37,000 tokens in the transcript from turn 1, replayed on every model call of a 20-plus-minute run.

The economics are exactly backwards. Carrying 37 K tokens across ~50 calls costs ~1.85 M token-reads; lazily re-expanding the one element being challenged costs ~1.5 K per item, paid once. The rule optimizes the cheap thing (a tool call) by maximizing the expensive thing (context residency).

It is also a pattern violation, not just a cost problem. “Keep bodies as your working copy” promotes the transcript to a **shadow source of truth**. Mid-run, the SME can edit an element in the web app, or `applyLint` can mutate it — the transcript copy is then stale, and the skill’s own rule tells the model to keep trusting it. Under the pattern, `expandElement` at the moment of need is the only correct read.

The fix (licensed directly by the architecture):

- Replace “read every body” with the cheap deterministic views that already exist: `getProcessSummary` (spine, counts, overview) + `getProcessRelations` (per-step relation/coverage map, ~1 KB per 20 steps, already ID-lists-only). These deliver the whole-process awareness the skill wants — “this step is referenced by an exception that contradicts its output” — without the bodies.
- Expand the `current` element and its directly related elements **lazily**, at the moment each item is challenged.
- Remove the “do not re-expand” instruction; fresh reads from the wiki are always correct, transcript memories are not.
- The skill already refuses to re-derive run facts by hand (`uncoveredSteps`, `currentHas-Control`, `outcomes_line`, `closeoutCounts` are deterministic tool outputs) — this change extends the same discipline to element content.

Session cycling, and why auto-compaction is the thing to avoid. When a long CLI session hits its context limit, Claude Code compacts — producing an *unprovenanced paraphrase* of wiki content the model then works from: a shadow wiki by another name. Because the foundational cursor, queue, and close-out facts all live in the runtime store (R9), the principled alternative is explicit: end the session every N queue items (or when cache-creation tokens spike), start fresh,

`getSessionStatus` → resume, re-read only the current item. Transcripts are disposable *because* the architecture keeps all durable state out of them — this is the Karpathy guardrail paying rent.

Same pattern, milder form:

- `document-ingest` (6.07 M cache-reads) holds the full source document plus every extracted element in context. Layer 1 is immutable, so re-reading the raw source per extraction batch is always safe by construction; extract per section with the conflict log in the runtime store.
 - The perspective specialists' "Phase 1 — Orientation" steps should orient from the summary/relation views rather than element bodies.
-

5. Lever 2 — Tool-Result Diet in the MCP Server

Findings from `src/lib/claude-mcp-server.ts` (~24 tools; benefits both tracks where output shapes are shared):

Finding	Location	Impact	Fix
<code>createElement</code> / <code>updateElement</code> echo the full element back	lines ~764, ~830	2–5 KB per write, duplicated into transcript and replayed every call thereafter	Return <code>{ok, id, title}</code> plus backend-generated facts only (generated ID, conformance warnings, <code>relations-Added</code>)
Every return uses <code>JSON.stringify(..., null, 2)</code>	all ~24 return sites	+40–60 % on every result, compounding through replay	Minify; models do not need indentation
<code>getProcessElements</code> returns full <code>meta</code> + <code>content</code> incl. the entire provenance map per element	line ~675	3–8 KB per collection read where 0.3–0.5 KB suffices	Abridged default (<code>id</code> , <code>title</code> , status flags) with <code>full: true</code> opt-in — mirroring what <code>expandElement</code> without an <code>id</code> already does correctly
No document-map equivalent on the MCP track	—	model falls back to coarse <code>getProcessSummary</code> or expensive full expands	Port the Gemini track's <code>generateDocumentMap</code> (99.5 % compression: 150 KB → ~1.5 KB) into a shared module; expose as a <code>getDocumentMap</code> tool

All of these are *fidelity-in-the-wrong-layer* corrections: the JSON document keeps every byte of provenance and evidence; the ephemeral view handed back to the model does not need it. Nothing about the stored document, the writers, or the conformance gates changes.

6. Lever 3 — Static Prompt Diet (Paid by Every Session, Every Call)

- **Skill frontmatter descriptions.** All 28 descriptions (~14.4 KB ≈ 3,600 tokens) sit in every Claude-track session's system prompt. Worst offenders run 540–670 characters (`dogfood-target` , `ntp-regenerate` , `council-review` , `dogfood-ntp`). Trim to ~200–250 characters — descriptions only need to win the routing decision.

- **Dogfood harnesses in production.** The three `dogfood-*` skills (50 KB of bodies, ~1.9 KB of descriptions) are loaded into production SME sessions despite their own text saying “Run ONLY when explicitly invoked in the CLI.” Move them out of `.claude/skills/` (e.g. a `dogfood/` directory wired up only for test runs).
 - **Skill-body redundancy (~30 KB across the set).** The Y/E/R loop, provenance read-back, and batching rules are restated in 8+ skills that `CORE_SYSTEM_PROMPT.md` already governs — replace with one-line references. The `WEB-PROVENANCE-BLOCK` is copy-pasted into the three `source-*` skills with a literal “keep in sync by hand” comment — hoist it into CORE §4.
 - **Schema restatements in prose.** Several skills (`transformation-agent`, `it-architect`, `document-ingest`) restate schema field structures (~2.4 KB total). These are forked copies of the layer-3 contract — both token waste and drift risk, the exact failure mode the schema-consistency test exists to prevent. Defer to `getSectionContext` / the schema.
 - **CLAUDE.md.** ~9.6 KB of developer documentation is loaded into every SME session worker (the CLI runs with the repo as cwd). The session model never needs the architecture-status note or verification commands. A leaner split is worth considering, traded against developer ergonomics.
-

7. Lever 4 — The Gemini Track Needs Its Own Structural Pass

`buildSystemInstruction` (`src/lib/gemini-worker.ts` ~803–907) injects **per model call**: the CORE prompt (8 KB) + **the entire raw** `schema/process-schema.json` (80 KB ≈ 20,000 tokens) + the active SKILL.md (3–20 KB) + the document map — and the tool loop re-sends this system instruction plus ~26 KB of tool declarations on every `generateContent` call, with no explicit caching and unbounded history growth.

Fixes, in order:

1. **Move the document map out of `systemInstruction` into the latest user message.** It is the only per-turn dynamic part; placing it at the front of the request defeats Gemini’s implicit prefix caching for everything behind it. A stable system prefix (CORE + schema slice + skill) makes implicit caching work; explicit cached-content for the prefix is the step beyond. The code already tracks `cachedContentTokenCount`, so the wiring half-expects this.
2. **Inject a per-skill schema slice, not the whole file.** Each skill touches a known set of element types (process-specialist → steps / roles / exceptions / pain-points / gaps / metrics). Derive the slice from `schema/process-schema.json` at injection time — the file remains the single source of truth, nothing is forked, the dual-representation drift guard is unaffected. Saves ~15,000 tokens per call.

3. **Cap tool-result size and history growth.** Tool responses are appended verbatim with no truncation and history is never windowed. The Lever-2 result diet helps automatically.

One bright spot worth keeping and reusing: `generateDocumentMap` / `customStringify` already achieve a 99.5 % reduction (150 KB process → ~1.5 KB map) with correct active-element expansion — this is the progressive-disclosure model done right, and it should become shared infrastructure for both tracks.

8. Lever 5 — Fix the Measurement First

`src/lib/token-usage.ts` `extractUsage` reads the CLI's `total_cost_usd` and the session route sums it **per turn**; the value is **cumulative per session**, so recorded dollar costs inflate roughly quadratically (~30x observed). Record per-turn deltas (track the previous cumulative value per session and subtract), verify whether the `usage` token fields are per-turn or cumulative, and add a test. Usage tallies stay in the runtime store — never the wiki.

This is cheap, and it makes every other improvement in this report measurable.

9. What This Plan Deliberately Does *Not* Do

To stay inside the schema and process-JSON principles:

- No trimming of provenance or evidence in the stored document — traceability is the product.
 - No abridged copies persisted anywhere; document maps and schema slices are computed on read, and even then never written into `wiki/`.
 - No bypassing of the typed writers; the mutation path is untouched.
 - No merging of the two schema representations (custom app schema vs. Draft-07 JSON Schema) — the drift-guard test continues to govern them.
-

10. Priority Order and Expected Impact

#	Action	Effort	Risk	Expected effect
1	Fix cost accounting (per-turn deltas)	Small	Low	Trustworthy measurements for everything below
2	Tool-result diet: no full-element echo, minify, abridged <code>getProcessElements</code>	Small	Low	Immediate cut on every skill, both tracks; ~10–25 KB less per session entering the transcript
3	Rewrite <code>foundational-run</code> Step 1 (orient from summary + relations, lazy expand, drop “don’t re-expand”); apply same to <code>document-ingest</code> and specialist Phase 1	Medium	Medium	Largest single win on measured data; removes the shadow-source-of-truth violation
4	Gemini track: doc map out of system instruction; per-skill schema slice; history caps	Medium	Medium	~15–20 K tokens saved per model call; prefix caching becomes effective
5	Static diet: trim descriptions, evict dogfood skills, dedupe skill boilerplate into CORE	Small	Low	~3–5 K tokens off every session turn
6	Session cycling for long runs via the runtime cursor; shared <code>getDocumentMap</code> tool on the MCP track	Medium	Medium	Caps worst-case context length; replaces auto-compaction with principled fresh reads

Items 1, 2 and 5 are low-risk mechanical changes. Items 3, 4 and 6 are the heavier ones and warrant an issue + impact + risk write-up each before development.

Prepared by Claude Code · Sources: `src/lib/claude-mcp-server.ts`, `src/lib/gemini-worker.ts`, `src/lib/session-worker.ts`, `src/lib/token-usage.ts`, `src/app/api/session/route.ts`, `.claude/skills/` (28 skills + `CORE_SYSTEM_PROMPT.md`), `schema/process-schema.json`, `data/runtime/*.json` recorded usage.